ソフトウェア情報学
基礎ゼミナールB
第８回

青森大学ソフトウェア情報学部
橋本 恭能
haship@aomori-u.ac.jp

目次

- 滑らかに走行する
  - ロボットの動かし方を変更する
  - 直進と旋回、直進と旋回を組合せる
  - 比例制御
- プログラムの追加・変更
  - 新しいメソッドの追加
  - 動作確認

ロボットの動かし方を変更

左右モータの回転方向・速度を
設定してロボットを動かす

▼

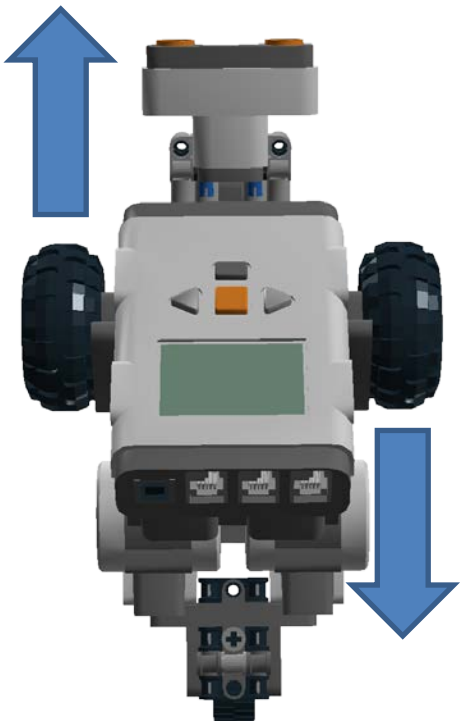
直進と旋回を組合せて
ロボットを動かす

直進と旋回

直進

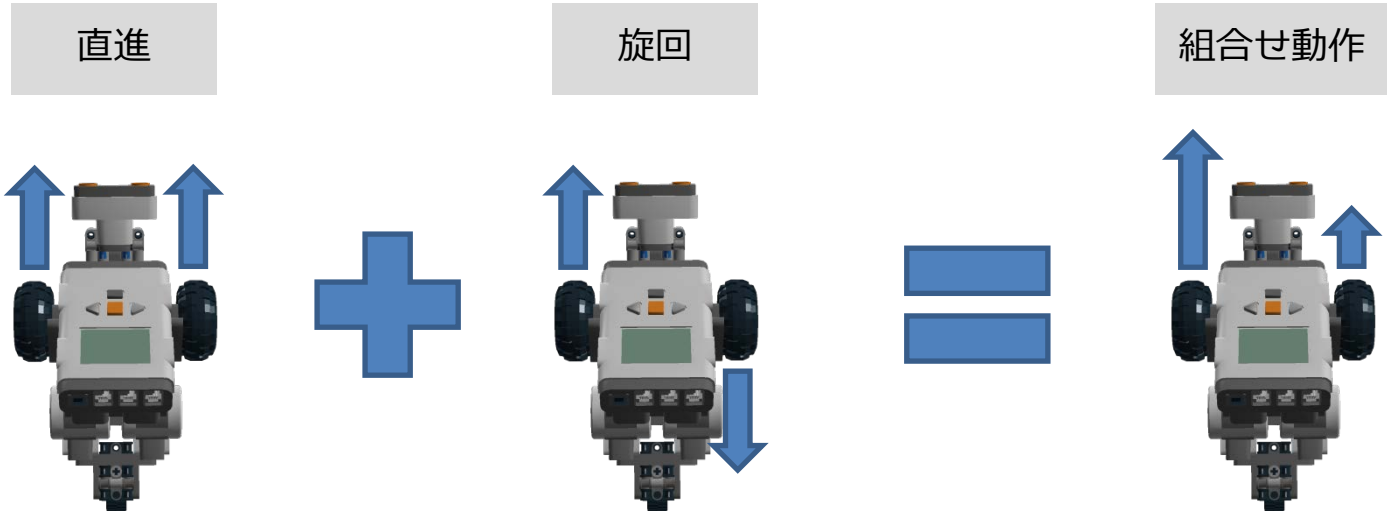
同じ速度・同じ方向

旋回



同じ速度・逆方向

直進と旋回を組合せる



直進の左モータパワー	＋	旋回の左モータパワー	＝	左モータのパワー
直進の右モータパワー	－	旋回の右モータパワー	＝	右モータのパワー

比例制御

- 目標値と測定値の差に比例した制御を行う
  - 白と黒の平均値を目標値とする
  - 光センサの値を測定値とする
  - （目標値ー測定値)×比例係数＝旋回のパワー
  - 左モータパワー＝直進パワー ＋ 旋回パワー
  - 右モータパワー＝直進パワー － 旋回パワー

プログラムの追加・変更

- 新しいメソッド追加
- ラインに沿って走行する

```
public void traceLine(){
    // ここに比例制御を使った
    // ライントレースの処理を記述
}
```

変更点①

```
import lejos.nxt.Button;
(以下省略)

public class LineTracer {

  private static final int THRESHOLD = 50;
  private static final float KP = 1.0F;
  private static final int FORWARD = 30;

  private NXTMotor leftMotor;
  (以下省略)

  public LineTracer(){
    (以下省略)
  }

  public static void main(String[] args) {
    (以下省略)
  }



}
```

点線部分を追加
比例制御の係数 KP
直進速度 FORWARD

```
  //ライン上にいるかどうか
  public boolean isOnLine(int val){
    return val < THRESHOLD;
  }
  //左ターン
  public void turnLeft(int power){
    (以下省略)
  }
  //右ターン
  public void turnRight(int power){
    (以下省略)
  }
  //停止
  public void stop(){
    (以下省略)
  }

  //ラインに沿って走行する
  public void traceLine(){


  }

}
```

点線部分を追加
新しいメソッド

変更点②

```
//ラインに沿って走行する
public void traceLine(){

  // 偏差 = 目標値(しきい値) - 測定値(センサー)
  int dev = THRESHOLD - light.readValue();

  // 旋回のパワー = 偏差 * 比例係数
  int turn = (int)(KP * dev);

  //  左モーターパワー = 直進 +  旋回
  leftMotor.setPower(FORWARD + turn);
  //  右モーターパワー = 直進 +  旋回
  rightMotor.setPower(FORWARD - turn);

  leftMotor.forward();
  rightMotor.forward();

}
```

ラインに沿って
走行する処理

```
public static void main(String[] args) {

  LineTracer01 robot = new LineTracer01();

  robot.stop();
  robot.light.setFloodlight(true);

  //タッチセンサーを押したら走行開始する
  while(!robot.touch.isPressed()){
   (以下省略)
  }

  while (Button.ESCAPE.isUp()) {

    robot.traceLine();

    //画面表示
    LCD.clear();
    LCD.drawInt( robot.light.readValue() , 0, 0);

    // 待機
    Delay.msDelay(100);
  }
  //robotを停止
  robot.stop();
}
```

点線部分を変更

点線部分を変更

変更後のプログラムリスト１

```
import lejos.nxt.Button;
import lejos.nxt.LCD;
import lejos.nxt.LightSensor;
import lejos.nxt.MotorPort;
import lejos.nxt.NXTMotor;
import lejos.nxt.SensorPort;
import lejos.nxt.TouchSensor;
import lejos.util.Delay;

public class LineTracer {

            private static final int THRESHOLD = 50;          //各自しきい値を変更する
            private static final float KP = 1.0F;             //比例ゲイン(各自調整)
            private static final int FORWARD = 30;            //直進速度

            private NXTMotor leftMotor;
            private NXTMotor rightMotor;
            private TouchSensor touch;
            private LightSensor light;

            public LineTracer(){
                        leftMotor = new NXTMotor(MotorPort.C);
                        rightMotor = new NXTMotor(MotorPort.B);
                        touch = new TouchSensor(SensorPort.S1);
                        light = new LightSensor(SensorPort.S3);
            }
```

変更後のプログラムリスト２

```
public static void main(String[] args) {

        LineTracer01 robot = new LineTracer01();

        robot.stop();
        robot.light.setFloodlight(true);

        //タッチセンサーを押したら走行開始する
        while(!robot.touch.isPressed()){
                //画面表示
                LCD.clear();
                LCD.drawInt(robot.light.readValue(), 0, 0);
                // 待機
                Delay.msDelay(100);
        }

        while (Button.ESCAPE.isUp()) {

                //ラインに沿って走行する
                robot.traceLine();

                //画面表示
                LCD.clear();
                LCD.drawInt(robot.light.readValue(), 0, 0);
```

変更後のプログラムリスト３

```
                        // 待機
                        Delay.msDelay(100);
            }
            //robotを停止
            robot.stop();
    }
    //ラインに沿って走行する
    public void traceLine(){
            int dev = THRESHOLD - light.readValue();
            int turn = (int)(KP * dev);

            leftMotor.setPower(FORWARD + turn);
            rightMotor.setPower(FORWARD - turn);

            leftMotor.forward();
            rightMotor.forward();
    }
    //ライン上にいるかどうか
    public boolean isOnLine(int val){
            return val < THRESHOLD;
    }
```

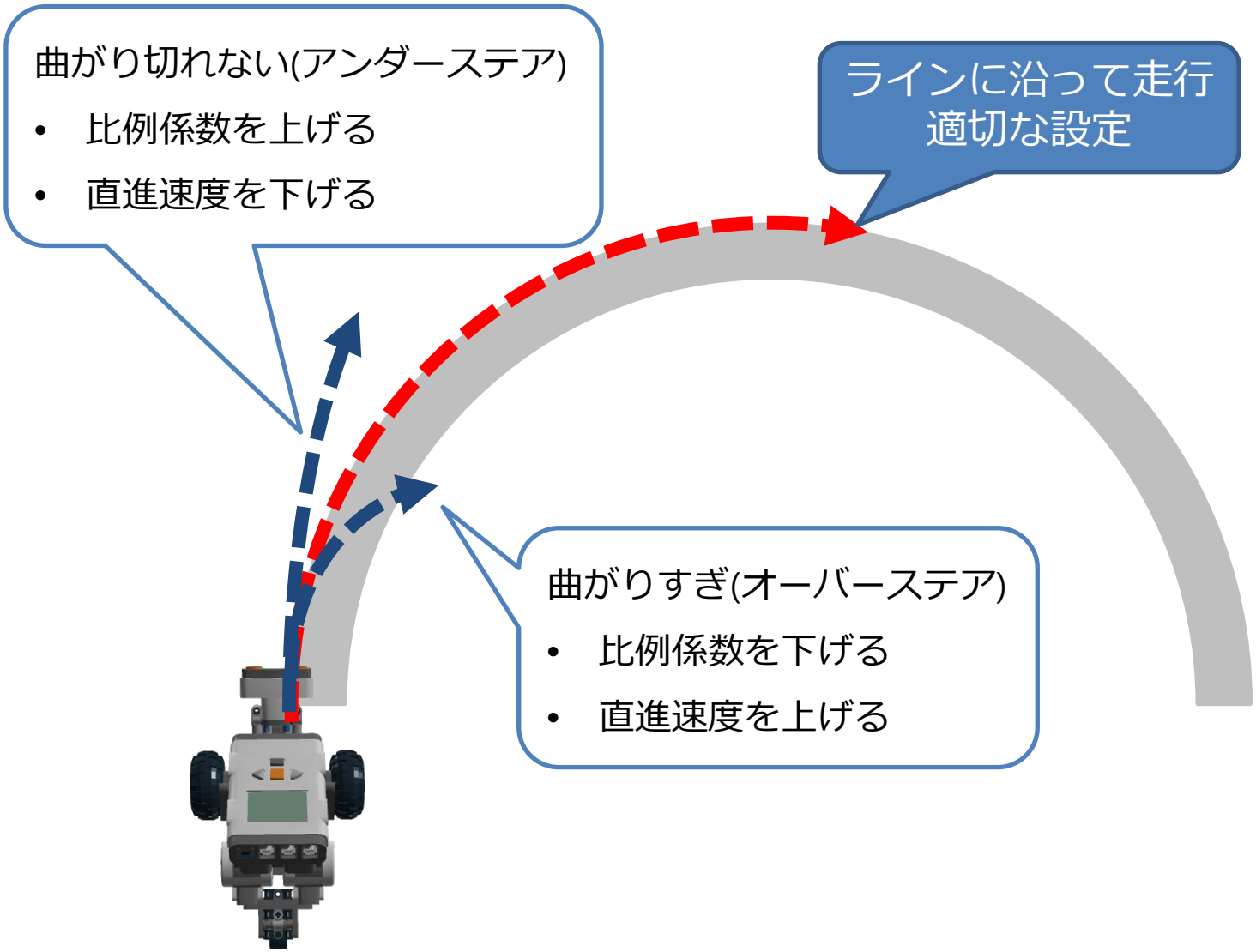
変更後のプログラムリスト４

```
        //左ターン
        public void turnLeft(int power){
                leftMotor.setPower(power);
                rightMotor.setPower(power);
                leftMotor.stop();
                rightMotor.forward();
        }
        //右ターン
        public void turnRight(int power){
                leftMotor.setPower(power);
                rightMotor.setPower(power);
                leftMotor.forward();
                rightMotor.stop();
        }
        //停止
        public void stop(){
                leftMotor.stop();
                rightMotor.stop();
        }
}
```

課題

- 変更したプログラムはまだ未調整です
- 以下の調整を行い、ラインに沿って走行できるようにする
  - 白・黒の値を測定してしきい値を計算し、THRESHOLDの値を変更する
  - 走行して旋回のパワーを上げ・下げするために、比例ゲインKPの値を調整する

動作確認


曲がり切れない(アンダーステア)
• 比例係数を上げる
• 直進速度を下げる
ラインに沿って走行
適切な設定
曲がりすぎ(オーバーステア)
• 比例係数を下げる
• 直進速度を上げる